

A Field Project Report On

PAISA

(FinTrack — A Personal Finance Tracker)

SUBMITTED BY

Safwan Ghare
Arham Sarang

Under the Guidance of

Prof. Kamil Khan

This Field Project Report is hereby presented to the esteemed Department of Computer Application, signifying the satisfactory completion of the requirements for the Semester V of Bachelor of Computer Application during academic year 2026-2027.

CERTIFICATE

This is to certify that Mr. Safwan Ghare and Mr. Arham Sarang, students of the Department of Computer Application, have successfully completed the Field Project titled "PAISA — Personal Finance Tracker" under the guidance of Prof. Kamil Khan during the academic year 2026-2027, in partial fulfilment of the requirements for the Semester V of the Bachelor of Computer Application programme.

The students actively participated in the requirement analysis, design, development, and testing of the system, and gained practical knowledge in the field of full-stack web application development, REST API design, and database-driven financial systems.

The project work carried out by the students has been found satisfactory and is in line with the objectives of the field project.

Project Guide
Prof. Kamil Khan

Head of Department

External Examiner

DECLARATION

We, Safwan Ghare and Arham Sarang, hereby declare that the Field Project Report titled "PAISA — Personal Finance Tracker" submitted in partial fulfilment of the requirements for the Semester V of the Bachelor of Computer Application is a record of original work carried out by us under the guidance of Prof. Kamil Khan.

The matter presented in this report has not been submitted, in part or in full, to any other university or institute for the award of any degree or diploma.

Safwan Ghare
Arham Sarang

Date: 23 September 2026

INDEX

SR. NO.	TOPIC
1	Introduction
2	Problem Definition
3	Existing System
4	Proposed System - Purpose - Scope & Objectives - Functional Requirements - Non-Functional Requirements
5	Need of the Proposed System
6	Scope of the Proposed System
7	Fact Finding Techniques
8	Feasibility Study
9	Hardware and Software Requirements
10	ER Diagram
11	Data Dictionary
12	DFD / UML Diagrams
13	Input / Output Screens
14	Limitations
15	Conclusion
16	Bibliography

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us in completing our Field Project titled "PAISA — Personal Finance Tracker".

First and foremost, we would like to thank our Project Guide, Prof. Kamil Khan, for providing valuable guidance, suggestions, and continuous support throughout the design and development of this project.

We are grateful to the Head of the Department and the college for providing us with the necessary facilities and encouragement to complete this project successfully.

We would also like to thank all the faculty members of the Department of Computer Application for their valuable support and suggestions during the project.

Finally, we express our heartfelt thanks to our parents, friends, and classmates for their encouragement and support throughout the completion of this project.

Thank You.

Project Team

Safwan Ghare and Arham Sarang

INTRODUCTION

PAISA is a full-stack personal finance tracking web application designed to help an individual user record income and expenses, organise them by category, plan monthly budgets, and stay on top of spending through automatic alerts. The system is built as a decoupled architecture: a React 19 (TypeScript, Vite, TanStack Query, shadcn/ui, Tailwind CSS) single-page front end that talks to a Node.js / Express 5 REST API, which in turn persists data in MongoDB using Mongoose 9.

The application supports secure authentication (JWT, delivered as an httpOnly cookie), role-based access (viewer, user, manager, admin), multi-currency support (INR, USD, EUR, GBP, JPY, AUD, CAD, SGD, AED), recurring transactions that auto-post on a schedule, CSV import/export, and a built-in "Paisa Assistant" that answers natural-language questions about the user's own finances using the Google Gemini API. A dedicated Budget Alert engine watches spending against each category's monthly limit and raises a warning at 80% and a danger alert at 100% of the limit.

This report documents the problem the project addresses, the system design (ER model, data dictionary, and UML/DFD diagrams), the functional and non-functional requirements, and the screens that make up the finished application.

PROBLEM DEFINITION

Most individuals track personal income and expenses using scattered methods — a mix of banking apps that only show one account, spreadsheets that require manual entry and manual charting, or no system at all. None of these give a single, categorised, real-time view of where money is going, and none of them proactively warn the user before a spending limit is crossed. As a result, overspending is usually noticed only after it happens, and recurring obligations (rent, subscriptions, salary) are tracked from memory rather than from a schedule.

EXISTING SYSTEM

Before a system like PAISA, a typical user relies on: (1) the transaction history built into a banking app, which is not categorised for budgeting and does not merge cash spending; (2) manually maintained spreadsheets, which require discipline to update and offer no automated alerting; or (3) generic note-taking, which has no structure at all. None of these approaches provide per-category budget limits, automatic 80%/100% threshold alerts, multi-currency support, or an assistant that can answer a plain-English question about spending patterns.

PROPOSED SYSTEM

The proposed PAISA platform provides a single, secure, multi-currency workspace where a user records every income and expense, classifies it by category, sets a monthly limit per category, and is automatically alerted as that limit is approached or crossed. Recurring income and expenses are scheduled once and auto-posted going forward, and an AI assistant gives instant, data-grounded answers about the user's own spending.

Purpose

To centralise personal financial record-keeping and replace manual tracking with an automated, categorised, and alert-driven system that gives the user a clear, current view of their income, expenses, and budget health at all times.

Scope & Objectives

The scope covers secure user registration and login, transaction CRUD with CSV import/export, global and custom categories, monthly per-category budgets with automatic 80%/100% alert detection, recurring transaction schedules with a materializer, multi-currency preferences, an admin layer for role management, and an AI assistant scoped to the logged-in user's own data. Real-time bank-account syncing, investment-portfolio tracking, and a native mobile app are out of scope for the current version.

Functional Requirements

- FR-01: The system shall allow a user to sign up and log in with an email and password, issuing a JWT held in an httpOnly cookie.
- FR-02: The system shall let a user create, view, edit, and delete income/expense transactions, each with an amount, type, category, date, and note.
- FR-03: The system shall let a user import transactions from a CSV file and export their transaction history to CSV.
- FR-04: The system shall provide a set of global default categories and allow the user to create, edit, and delete their own custom categories.
- FR-05: The system shall let a user set a monthly spending limit (budget) per expense category and automatically detect when spending reaches 80% or 100% of that limit.
- FR-06: The system shall notify the user of triggered budget alerts and let them acknowledge one alert or all alerts.
- FR-07: The system shall let a user define recurring transaction schedules (daily/weekly/monthly/yearly) that auto-post a real transaction on their due date.
- FR-08: The system shall let a user switch their preferred display currency among nine supported currencies.
- FR-09: The system shall provide an AI assistant endpoint that answers a user's natural-language question using only that user's own recent transactions, budgets, and categories as context.
- FR-10: The system shall let an administrator view all users and their lifetime totals, and

promote or demote a user's role.

Non-Functional Requirements

- Performance: List endpoints support filtering, sorting, field projection, and pagination (capped at 100 records per page) so responses stay fast as data grows.
- Security: Passwords are hashed with bcrypt and never returned by the API; every data route is scoped to the authenticated user's own records so one user can never read or modify another's data.
- Usability: The interface is built with shadcn/ui and Tailwind CSS for a clean, responsive, accessible experience across desktop and mobile.
- Maintainability: The backend follows a layered (routes -> controllers -> models) architecture with a single global error handler, keeping the codebase easy to extend.
- Reliability: Budget alerts use a unique compound index on (user, category, month, threshold) so re-running detection can never create a duplicate alert.

NEED OF THE PROPOSED SYSTEM

A dedicated system like PAISA is needed because manual or fragmented finance tracking leaves the user unaware of overspending until after it happens, and gives no structured way to plan monthly spending by category. PAISA is needed for the following reasons:

- **Centralised Record-Keeping:** Every income and expense, whatever the source, is recorded in one categorised ledger instead of being scattered across apps, receipts, and memory.
- **Proactive Budget Control:** Automatic 80%/100% threshold detection warns the user before a category is over budget, not after.
- **Recurring Obligation Tracking:** Rent, subscriptions, EMIs, and salary are scheduled once and auto-posted, removing the risk of a forgotten recurring entry.
- **Multi-Currency Support:** A user who deals in more than one currency can record and view their finances in the currency that makes sense to them.
- **Instant Insight:** The Paisa Assistant answers plain-English questions ("How much did I spend on food this month?") without the user needing to build a report manually.
- **Data Portability:** CSV import/export means a user is never locked into the platform and can migrate data in or out at will.
- **Secure, Private Access:** JWT-based authentication and per-user data scoping ensure a user's financial data is visible only to them.

SCOPE OF THE PROPOSED SYSTEM

The current scope of PAISA focuses on individual personal finance management. It includes:

- User Management and Authentication: Registration, login, logout, password reset via emailed token, and profile updates (name, email, currency).
- Transaction Management: Full CRUD on income/expense transactions, spending-by-category and income/expense/balance summary aggregations, and CSV import/export.
- Category Management: Global default categories plus user-owned custom categories, each unique per (user, name, type).
- Budget & Alert Engine: Per-category monthly limits, automatic 80%/100% breach detection, and an alert inbox with acknowledge/unread-count support.
- Recurring Transactions: Schedule templates with configurable frequency and interval, a materializer that posts all due transactions, and pause/resume support.
- AI Assistant: A read-only, Gemini-backed assistant that answers questions about the user's own recent transactions, budgets, and categories.
- Admin Layer: A dedicated admin role that can list all users with their lifetime totals and promote/demote roles.

Potential Applications

- Individual users tracking personal or household income and expenses.
- Students and young professionals building a monthly budgeting habit.
- Freelancers who need to separate multiple income sources and currencies.

Exclusions (Future Possibilities)

The current version does not include direct bank-account or card syncing, investment-portfolio tracking, shared/family accounts, push notifications, or a native mobile app. These remain potential future enhancements.

FACT FINDING TECHNIQUES

1. Interview

Informal discussions were held among the project team to clarify the exact set of financial events (income, expense, recurring bill, budget breach) a personal finance app must model, which shaped the transaction, budget, and alert schema.

2. Observation

Common personal budgeting habits — spreadsheets, banking-app statements, and mental tracking — were studied to identify their gaps, which directly motivated the automatic budget-alert and recurring-transaction features.

3. Questionnaires

Informal feedback from prospective users was gathered on which categories, currencies, and alert thresholds would be most useful, guiding the choice of the default category list and the 80%/100% thresholds.

4. Record Searching

Documentation for comparable personal-finance tools and for the chosen technology stack (Express, Mongoose, TanStack Query, Google Gemini API) was reviewed to align the implementation with established patterns and API conventions.

FEASIBILITY STUDY

1. Technical Feasibility

The project uses a modern, well-documented stack: React 19 with TypeScript and Vite on the front end, and Express 5 with Mongoose 9 on MongoDB for the back end. JWT handles stateless authentication, and the layered controller/model architecture keeps each concern isolated, making the system straightforward to build, test, and extend with a small team.

2. Economic Feasibility

All core technologies — React, Express, MongoDB, Mongoose — are open source and free to use. The only paid dependency is optional: the Google Gemini API for the AI assistant, which is usage-based and can be left unconfigured without breaking the rest of the application. This keeps the project economically viable for an academic or personal deployment.

3. Operational Feasibility

The application is designed to be usable without training: the dashboard surfaces totals and alerts immediately after login, forms are validated inline, and the sidebar badge highlights unacknowledged alerts. This makes day-to-day operation straightforward for a non-technical user.

4. Schedule Feasibility

The project was developed within a single-semester academic timeframe, following a structured progression: schema and API design, backend implementation (auth, transactions, budgets, alerts, recurring engine), frontend implementation, AI assistant integration, and finally testing and this documentation.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirement

To support development and smooth operation of the application:

- Processor: Multi-core processor (Intel Core i5 or AMD Ryzen 5 recommended).
- RAM: 8 GB recommended (4 GB minimum for basic testing).
- Storage: Minimum 2 GB free disk space for the development environment and dependencies.
- Network: Stable internet connection for the MongoDB Atlas connection and the Gemini API.

Software Requirement

The project relies on the following development and runtime environments:

- Operating System: Windows 10/11, macOS, or Linux.
- Backend: Node.js 20+, Express 5, Mongoose 9, JWT, bcryptjs, Nodemailer.
- Frontend: React 19, TypeScript 5.8, Vite 8, React Router 7, TanStack Query, Tailwind CSS v4, shadcn/ui (Radix UI), Recharts.
- Database: MongoDB (local or MongoDB Atlas).
- External Services: Google Gemini API (AI assistant), SMTP email service (password reset).
- Development Tools: Visual Studio Code, Postman/Thunder Client, a modern web browser (Chrome/Edge/Firefox).

ER DIAGRAM

The data model centres on the User. Each User owns a set of Transactions, may define Custom Categories (in addition to the platform's Global Categories), sets monthly Budgets per category, receives Budget Alerts when a budget is breached, and can schedule Recurring Transactions that auto-post real Transactions on a due date.

Entities and Relationships

- USER (1) --- creates ---> (N) CATEGORY [custom categories only; global categories have user = null]
- USER (1) --- records ---> (N) TRANSACTION
- CATEGORY (1) --- classifies ---> (N) TRANSACTION
- USER (1) --- sets ---> (N) BUDGET
- CATEGORY (1) --- limits ---> (N) BUDGET
- USER (1) --- receives ---> (N) BUDGET_ALERT
- CATEGORY (1) --- triggers ---> (N) BUDGET_ALERT
- USER (1) --- schedules ---> (N) RECURRING_TRANSACTION
- CATEGORY (1) --- classifies ---> (N) RECURRING_TRANSACTION

A compound unique index on (user, category, month_year) keeps a Budget one-per-category-per-month, and a compound unique index on (user, category, month_year, threshold) keeps a Budget Alert one-per-threshold-per-category-per-month, so re-running alert detection is idempotent.

DATA DICTIONARY

The system persists six MongoDB collections, managed through Mongoose schemas.

11.1 users

Field	Type	Key / Constraint	Ref	Description
_id	ObjectId	Primary Key	-	Unique user identifier
name	String	Required	-	Full name of the user
email	String	Unique, Required	-	Login email (lowercased)
password	String	Required, hidden	-	Bcrypt password hash
currency	String (enum)	Default INR	-	Display currency, 9 supported
role	String (enum)	Default user	-	viewer / user / manager / admin
active	Boolean	Default true, hidden	-	Soft-delete flag

11.2 categories

Field	Type	Key / Constraint	Ref	Description
_id	ObjectId	Primary Key	-	Unique category identifier
name	String	Required, max 50	-	Category label
type	String (enum)	Required	-	income / expense
user	ObjectId	Nullable	User	null = global default category

11.3 transactions

Field	Type	Key / Constraint	Ref	Description
_id	ObjectId	Primary Key	-	Unique transaction identifier
user	ObjectId	Required	User	Owning user

category

ObjectId

Nullable

Category

				null = uncategorized
amount	Number	Required, > 0	-	Always stored positive
type	String (enum)	Required	-	income / expense
note	String	Optional, max 255	-	Free-text note
date	Date	Required	-	Transaction date

11.4 budgets

Field	Type	Key / Constraint	Ref	Description
_id	ObjectId	Primary Key	-	Unique budget identifier
user	ObjectId	Required	User	Owning user
category	ObjectId	Required	Category	Target expense category
month_year	String	Required, 'YYYY-MM'	-	Budget month
limit_amount	Number	Required, > 0	-	Monthly spending limit

11.5 budget_alerts

Field	Type	Key / Constraint	Ref	Description
_id	ObjectId	Primary Key	-	Unique alert identifier
user	ObjectId	Required	User	Owning user
category	ObjectId	Required	Category	Category that breached
month_year	String	Required, 'YYYY-MM'	-	Alert month
threshold	Number (enum)	Required	-	80 or 100 (percent)
acknowledged	Boolean	Default false	-	Seen/dismissed by user
emailed	Boolean	Default false	-	Alert email already sent

11.6 recurring_transactions

Field	Type	Key / Constraint	Ref
-------	------	------------------	-----

				Description
_id	ObjectId	Primary Key	-	Unique schedule identifier
user	ObjectId	Required	User	Owning user
category	ObjectId	Nullable	Category	null = uncategorized
type / amount	String / Number	Required	-	Direction and amount to post
frequency	String (enum)	Required	-	daily / weekly / monthly / yearly
interval_count	Number	Default 1	-	e.g. every 2 weeks
next_run	String	'YYYY-MM-DD'	-	Next date due to post
end_date	String	Optional	-	Schedule stops after this date
active	Boolean	Default true	-	false = paused / ended

Database Relationships

- User to Transaction / Budget / Budget Alert / Recurring Transaction / Category: one-to-many, scoped by the "user" foreign key on every collection.
- Category to Transaction / Budget / Budget Alert / Recurring Transaction: one-to-many, via the "category" reference.
- Uniqueness: (user, category, month_year) is unique on budgets; (user, category, month_year, threshold) is unique on budget_alerts; (user, name, type) is unique on categories.

DFD / UML DIAGRAMS

12.1 Data Flow Diagram — Level 0 (Context Diagram)

The Context Diagram shows PAISA as a single process exchanging data with three external entities: the Registered User (and Admin), the Google Gemini API (AI assistant), and the SMTP Email Service (password-reset emails), all backed by the MongoDB database.

12.2 DFD Level 1

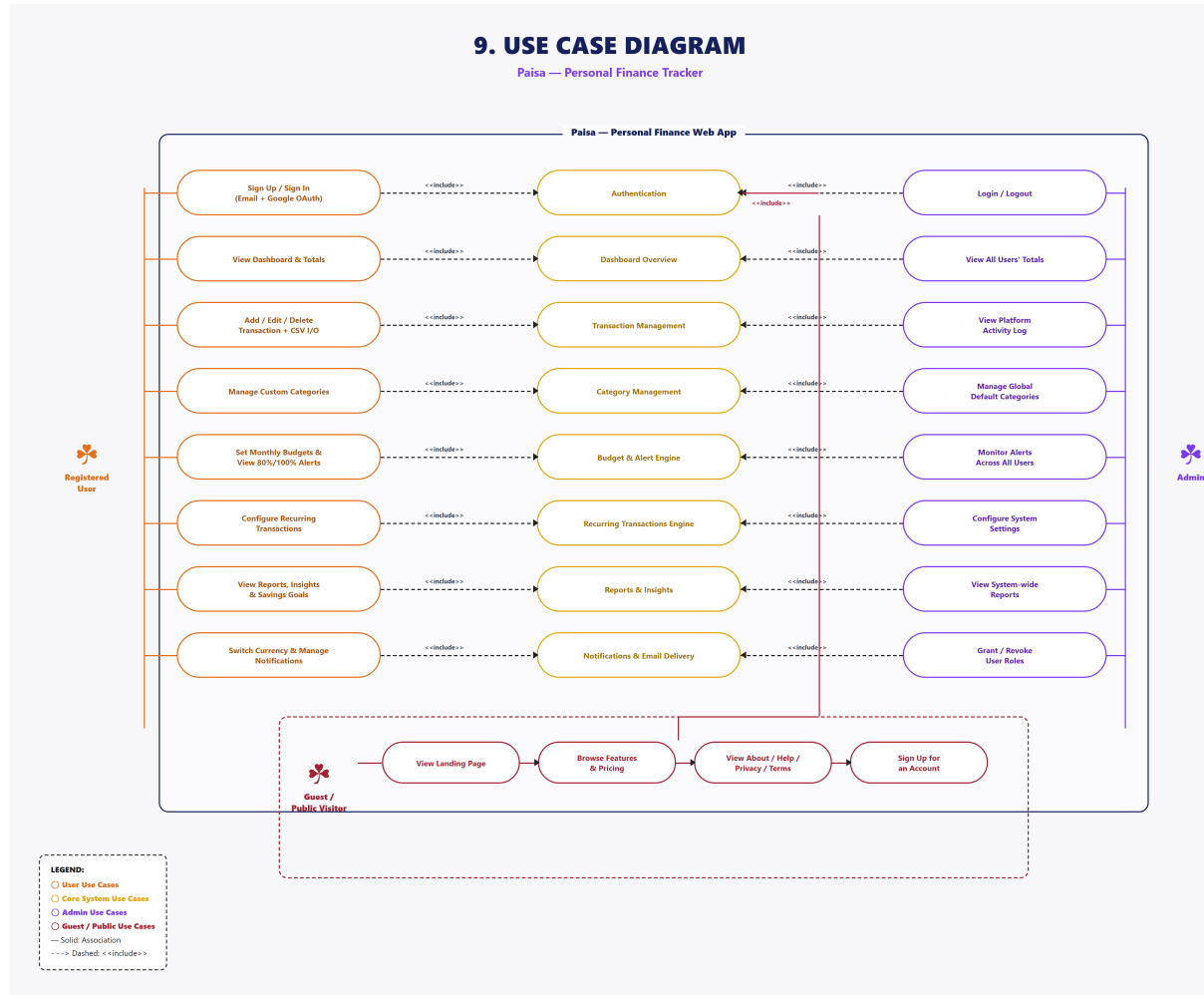
The Level 1 DFD decomposes PAISA into eight processes: (1) User Authentication & Account Management, (2) Category Management, (3) Transaction Management, (4) Budget Management, (5) Budget Alert Engine, (6) Recurring Transaction Engine, (7) AI Assistant, and (8) Admin & Role Management — each reading from and writing to its own MongoDB collection (D1 Users, D2 Categories, D3 Transactions, D4 Budgets, D5 Budget Alerts, D6 Recurring Transactions).

13. UML DIAGRAMS

UML (Unified Modeling Language) diagrams are used to represent the structure, behaviour, interactions, and deployment architecture of the PAISA platform.

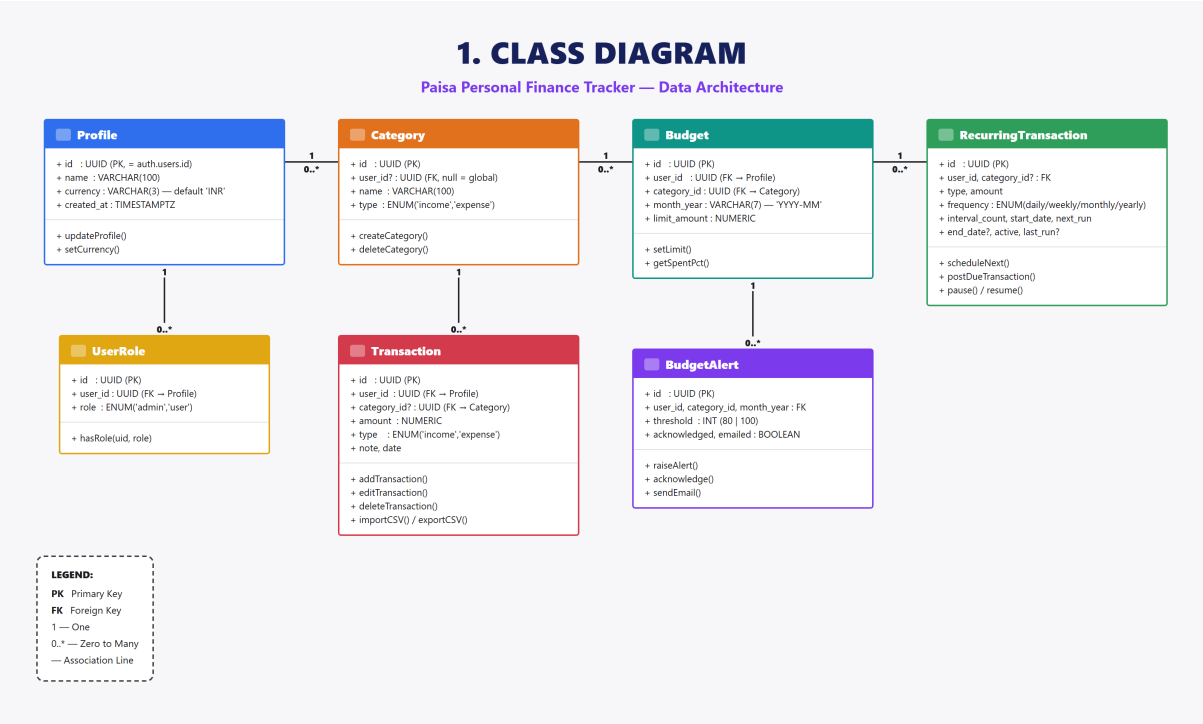
13.1 Use Case Diagram

The Use Case Diagram represents the functional interactions between the system and its actors: the Registered User, the Admin, and a Guest/Public visitor, alongside the external Authentication, Budget & Alert Engine, and Notification systems.



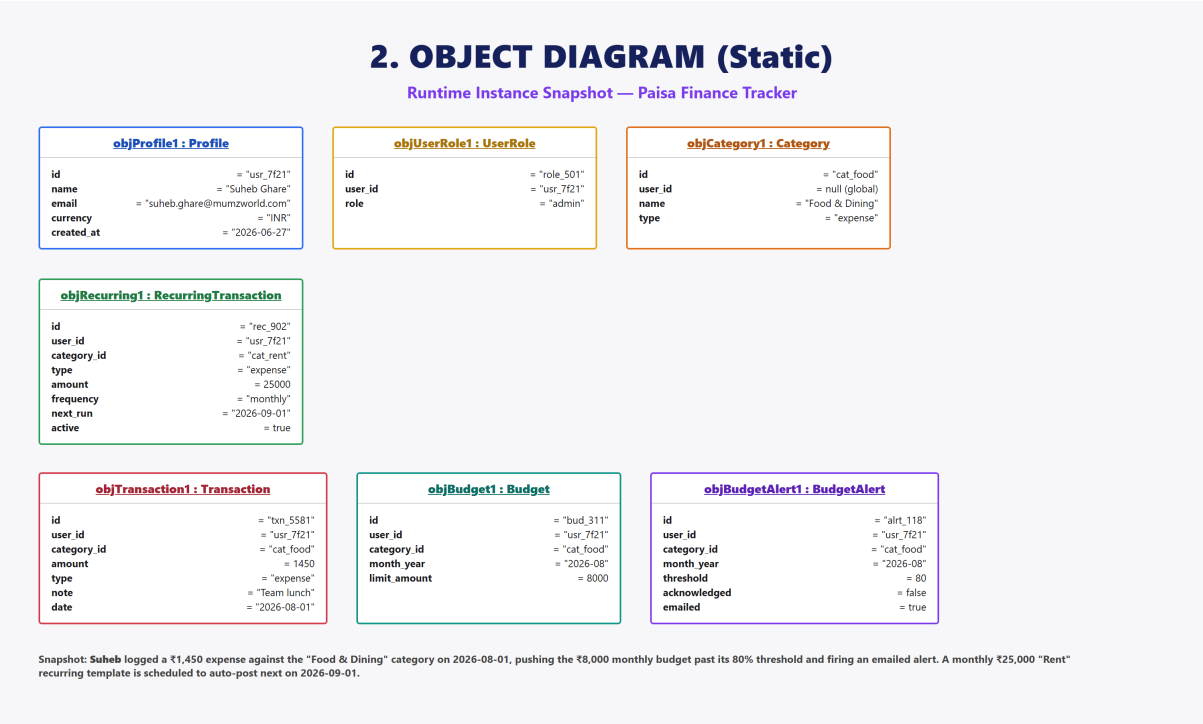
13.2 Class Diagram

The Class Diagram represents the static structure of PAISA. It identifies the major classes — User, UserRole, Category, Transaction, Budget, BudgetAlert, and RecurringTransaction — along with their attributes, methods, and relationships.



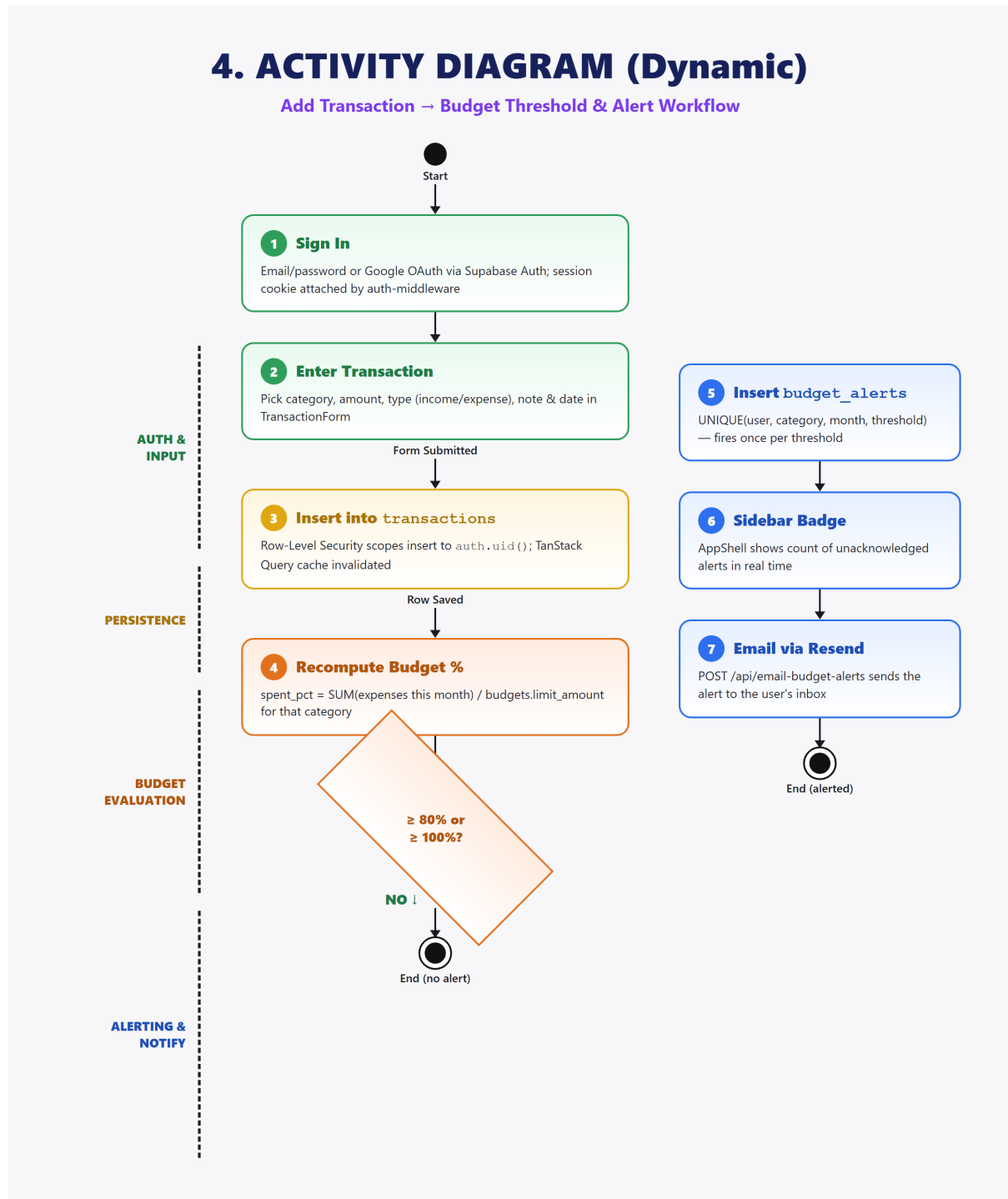
13.3 Object Diagram

The Object Diagram represents a snapshot of objects and their relationships at a point in time, illustrating representative (sample) instances of User, Category, Transaction, and Budget.



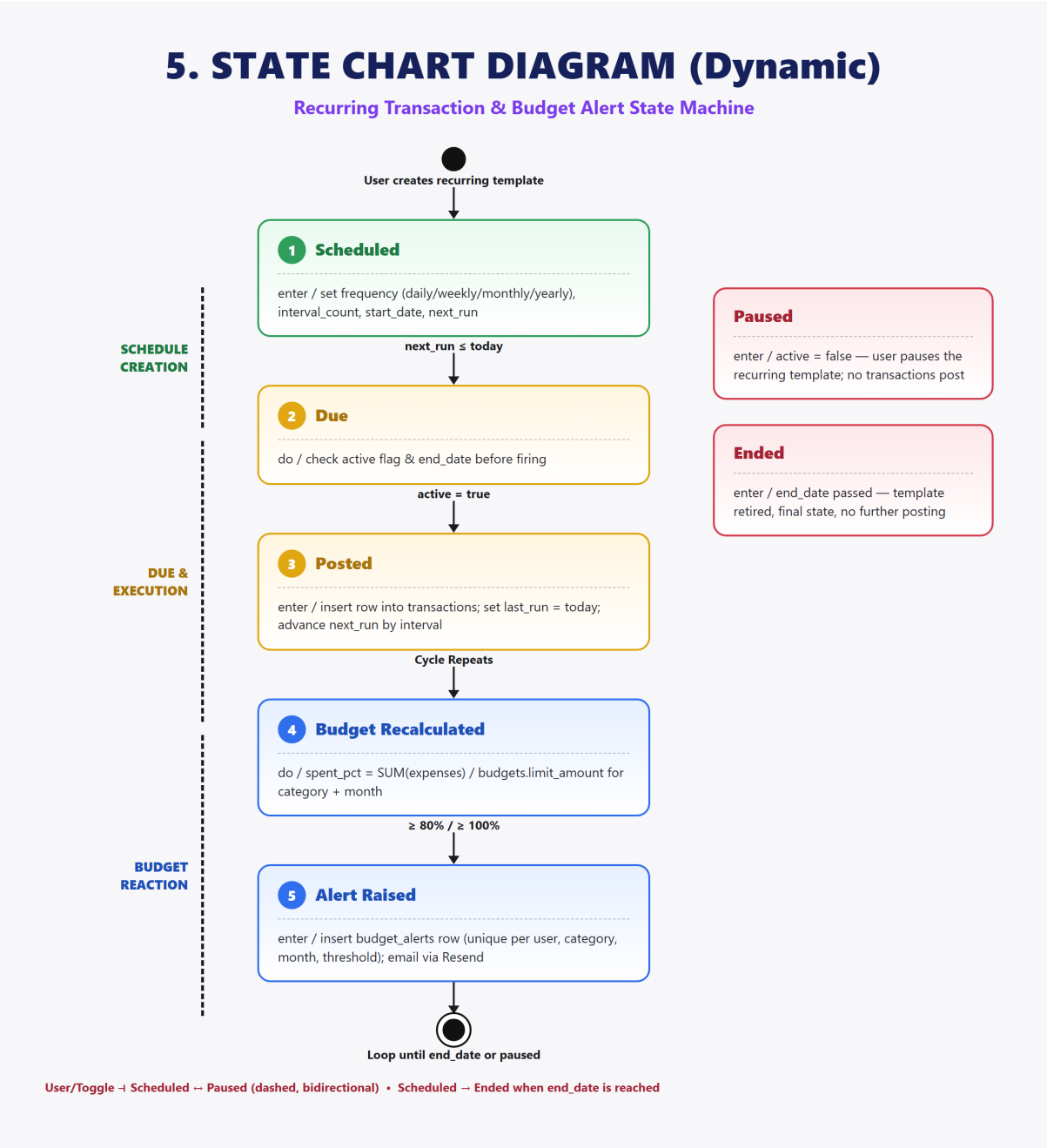
13.4 Activity Diagram

The Activity Diagram represents the workflow of adding a transaction: from login, through form submission and validation, to the budget-recalculation and alert-detection step that follows a successful save.



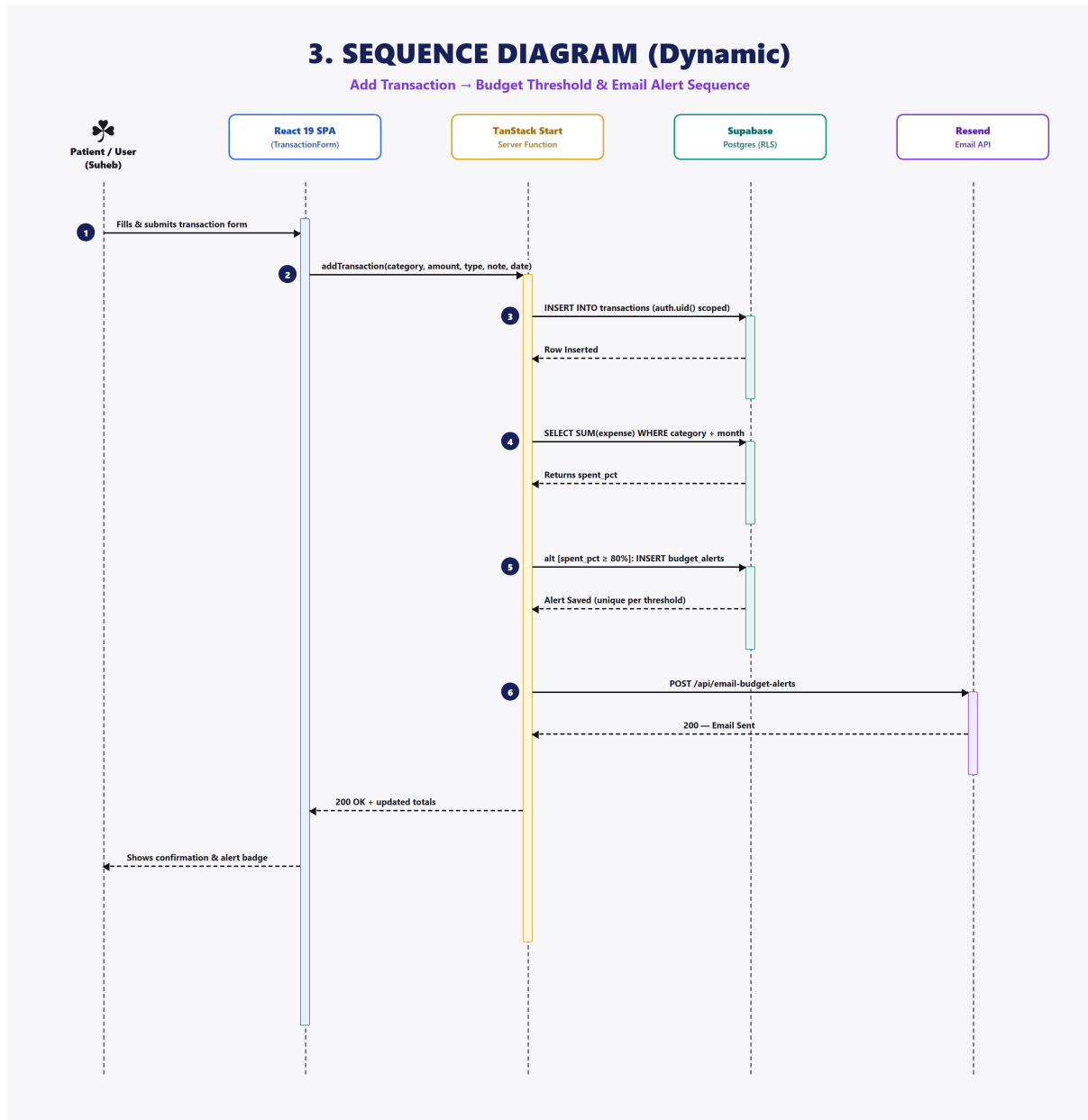
13.5 State Chart Diagram

The State Chart Diagram represents the lifecycle of a Budget Alert — from creation when a threshold is crossed, through the unacknowledged state, to acknowledgement by the user.



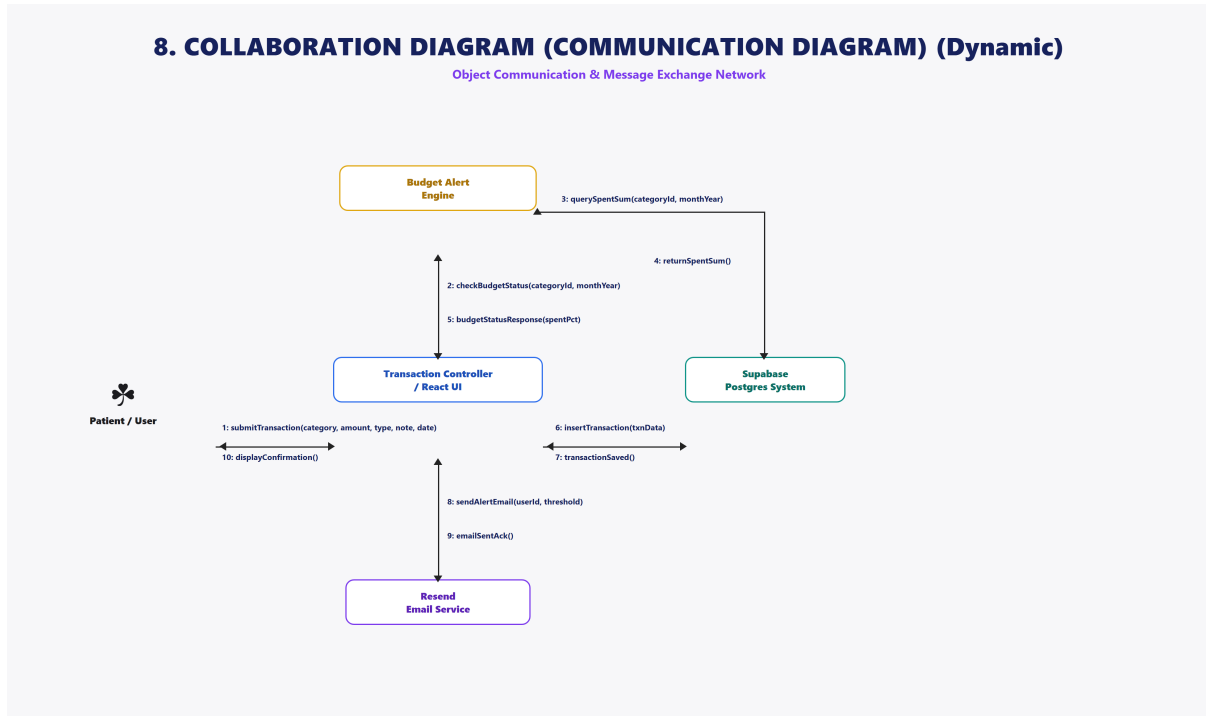
13.6 Sequence Diagram

The Sequence Diagram represents the chronological interaction between the User, the React frontend, the Express API, and MongoDB during the add-transaction and budget-alert-detection workflow.



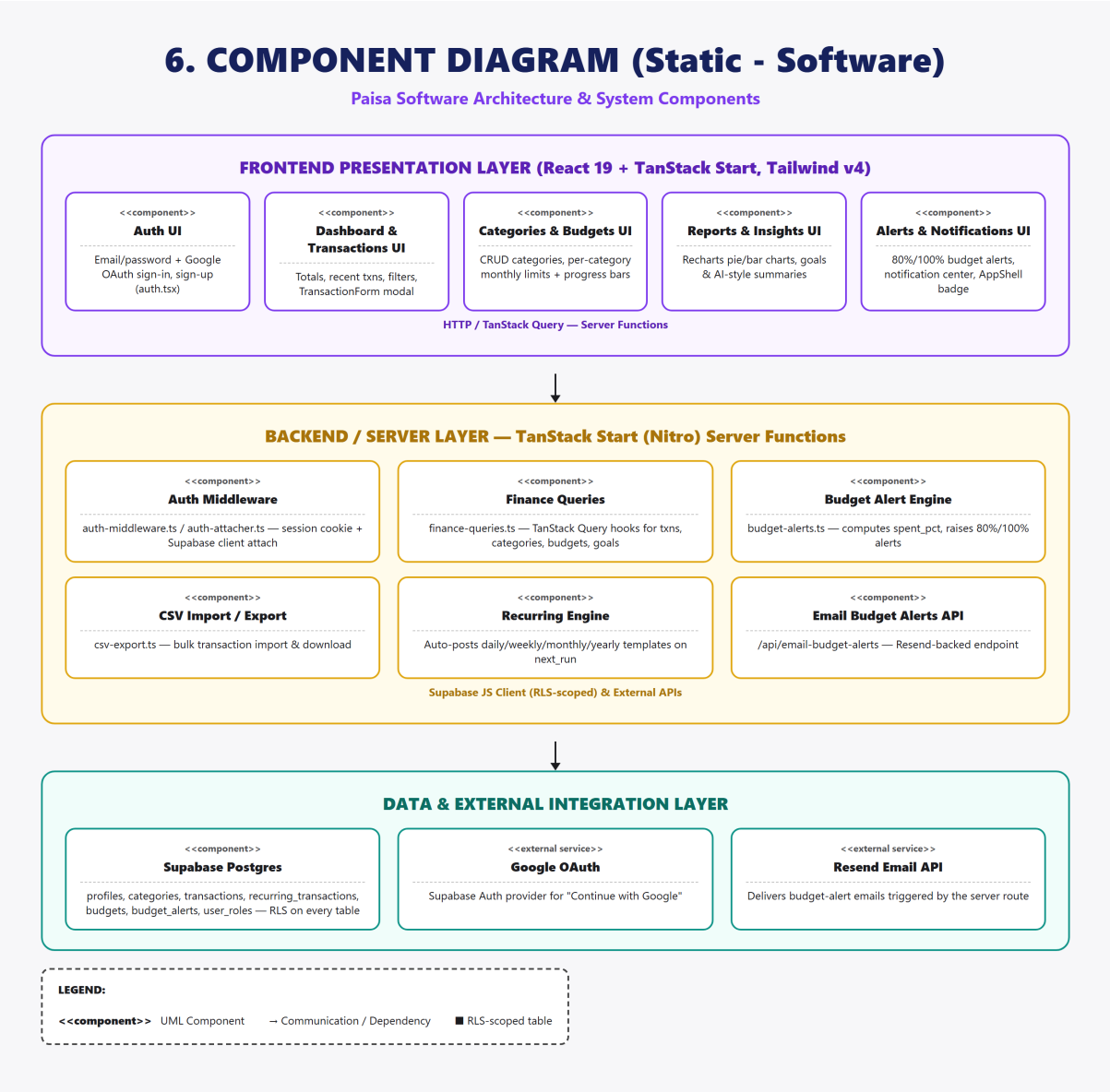
13.7 Collaboration / Communication Diagram

The Collaboration Diagram represents how the User, Frontend, Express Backend, and MongoDB objects communicate with one another to complete the add-transaction and budget-alert operation.



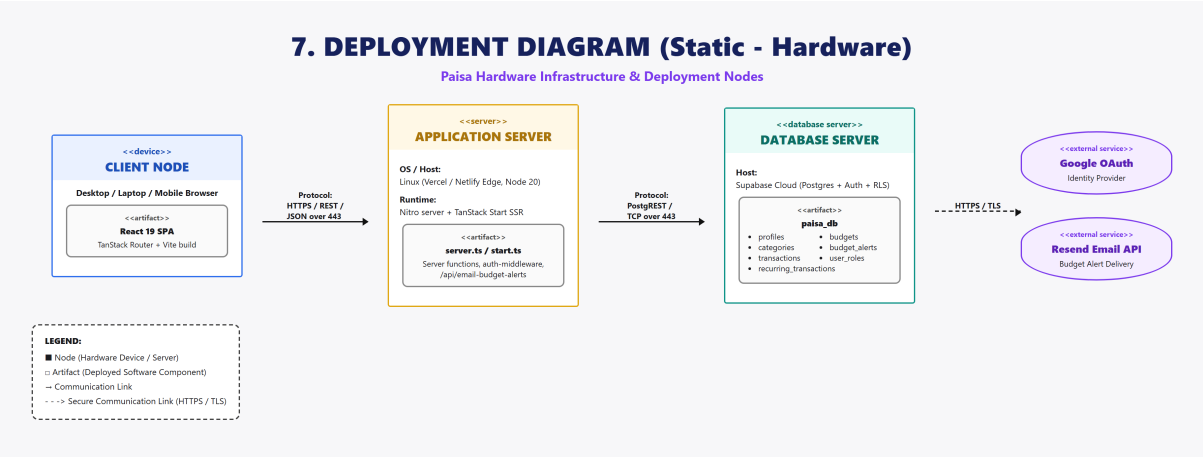
13.8 Component Diagram

The Component Diagram represents the high-level software architecture of PAISA: the React frontend, its TanStack Query data layer, the Express backend (routes, controllers, models), MongoDB, and the external Gemini and SMTP services.



13.9 Deployment Diagram

The Deployment Diagram represents the physical and logical deployment architecture: the client browser running the React SPA, the Node.js/Express application server, the MongoDB database server, and the external Google Gemini and SMTP services reached over HTTPS.



INPUT / OUTPUT SCREENS

This section catalogs the primary user interfaces of the PAISA application.

Screen Name	Purpose	User	Key Functionality
Landing Page	Marketing entry point	All	Navigation, sign up
Dashboard	Monthly overview	User	Totals, budget alerts, charts
Transactions	Manage all entries	User	Filter, add, edit, CSV import/export
Recurring	Manage schedules	User	Create, pause, resume schedules
Calendar	Upcoming due items	User	Monthly recurring-bill view
Categories	Manage categories	User	Add custom, view defaults
Budgets	Set monthly limits	User	Progress bars, breach warnings
Reports	Visualise spending	User	Pie / bar charts, monthly trend

Figure 13.1: Landing Page

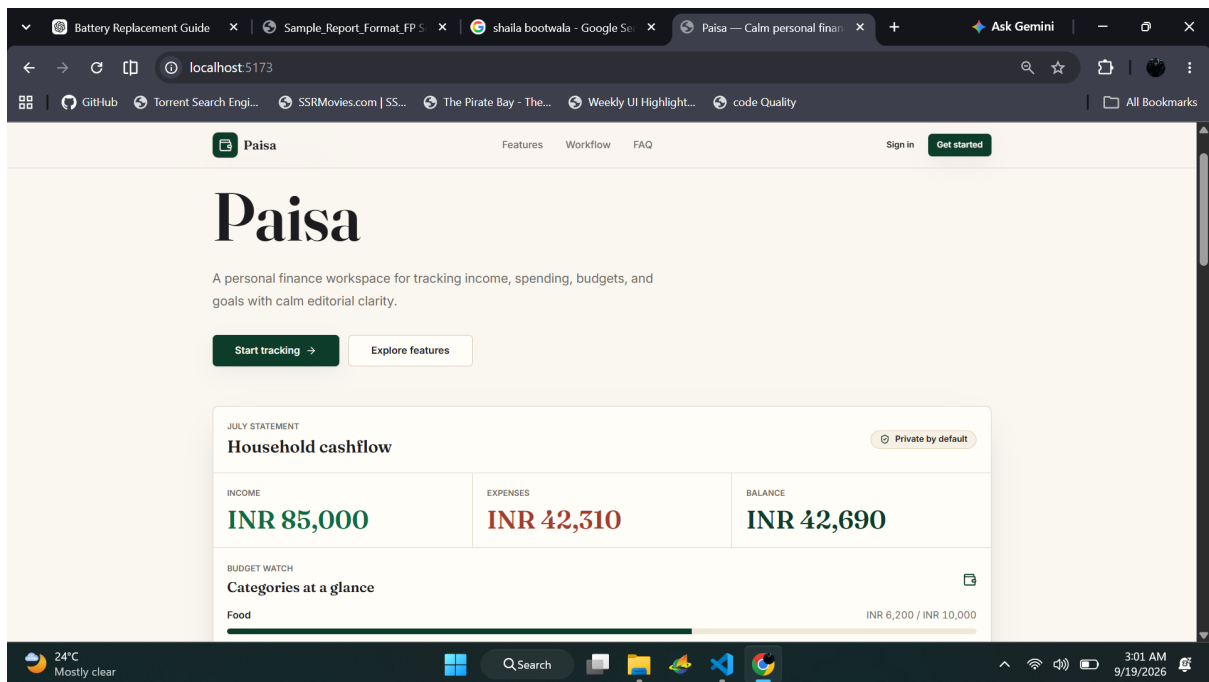


Figure 13.2: Dashboard

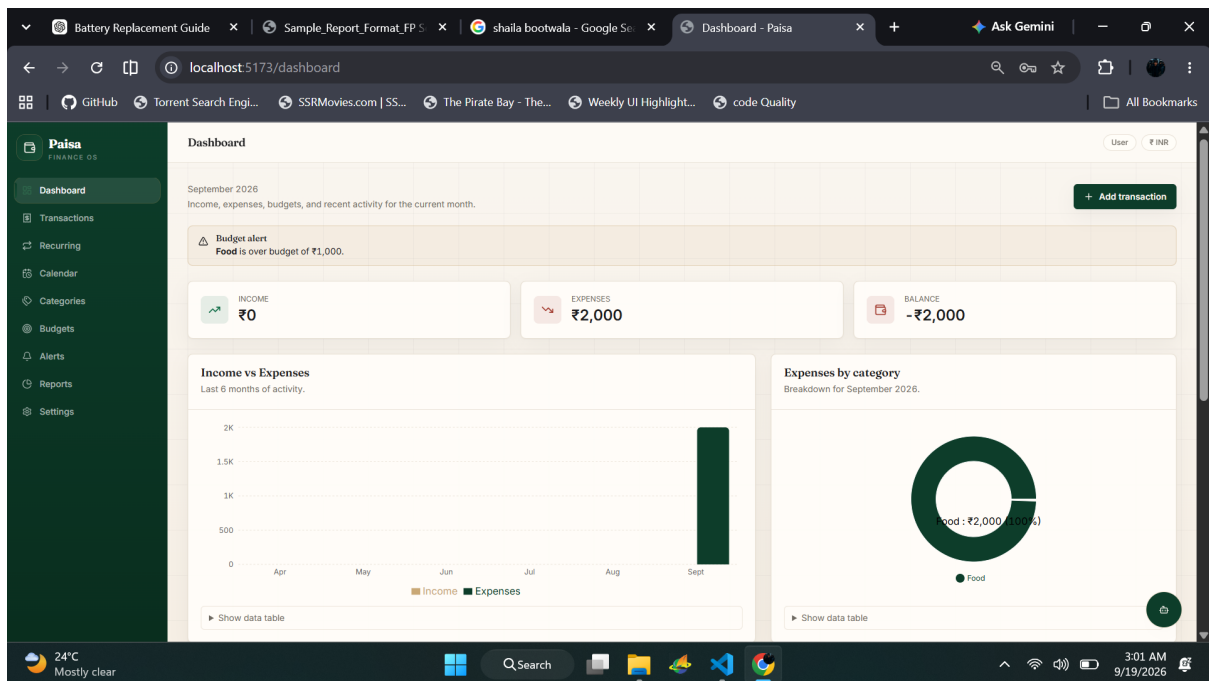


Figure 13.3: Transactions

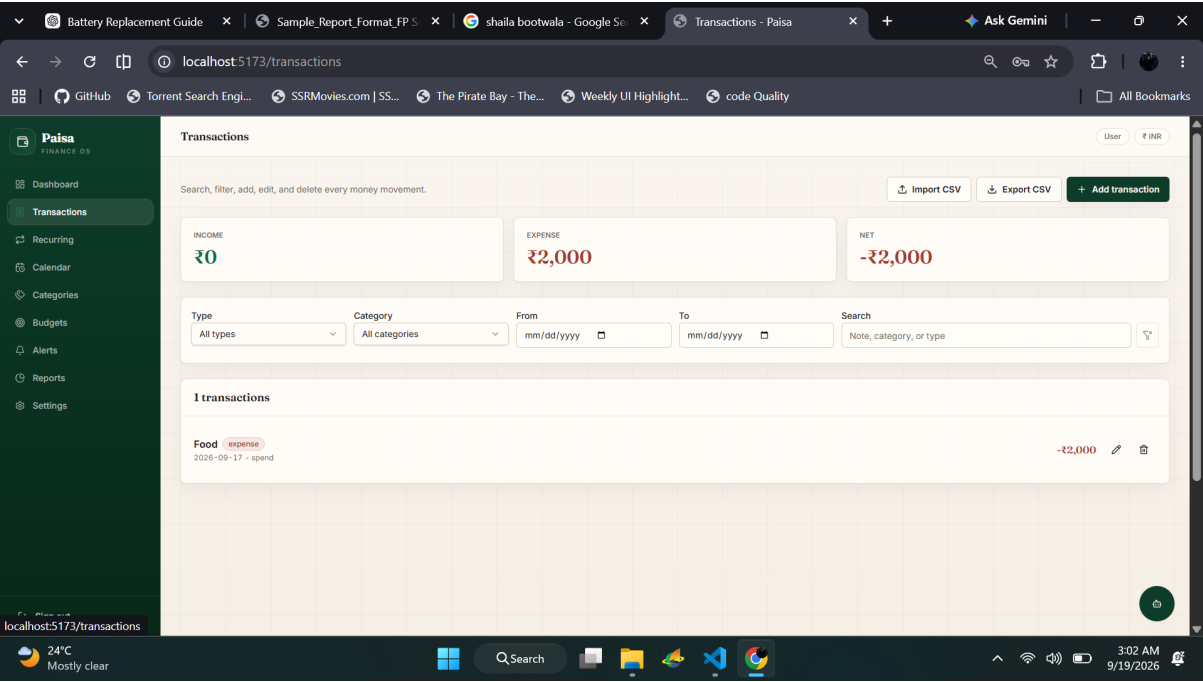


Figure 13.4: Recurring Transactions

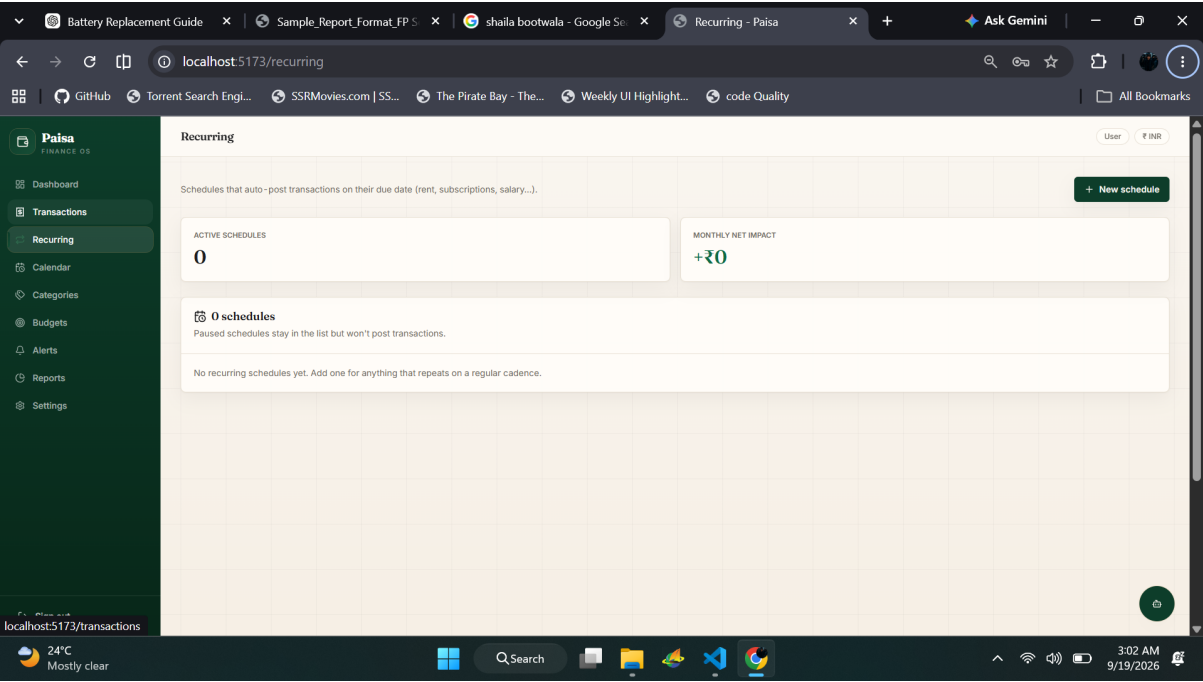


Figure 13.5: Calendar

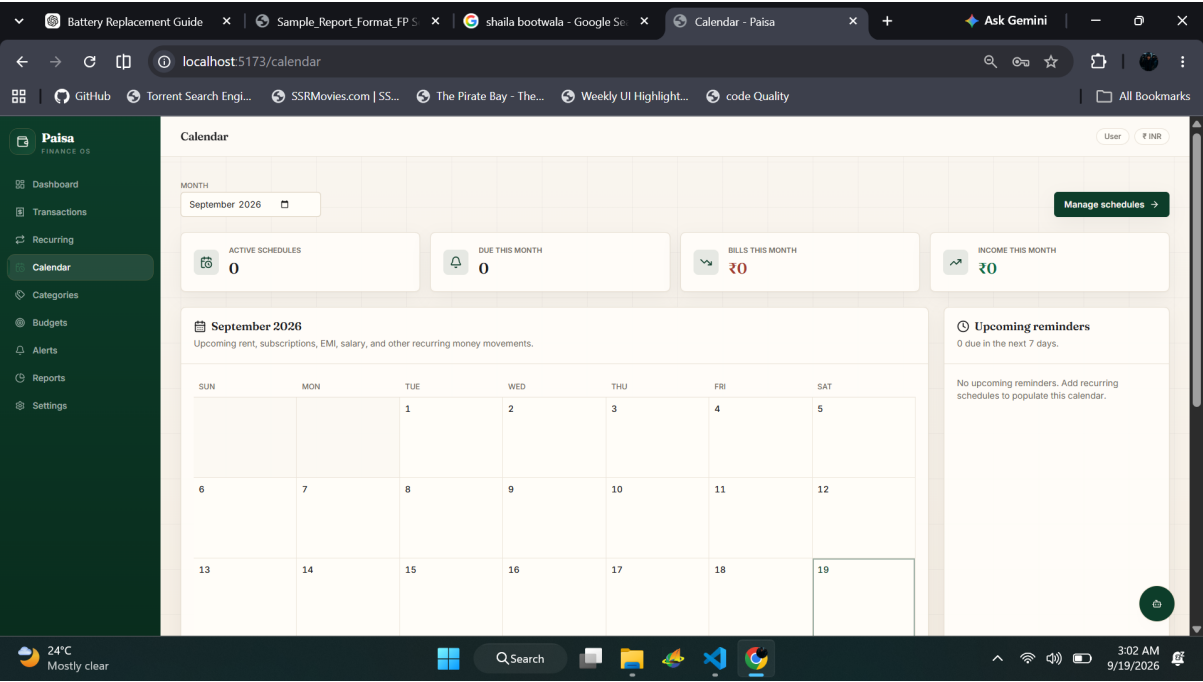


Figure 13.6: Categories

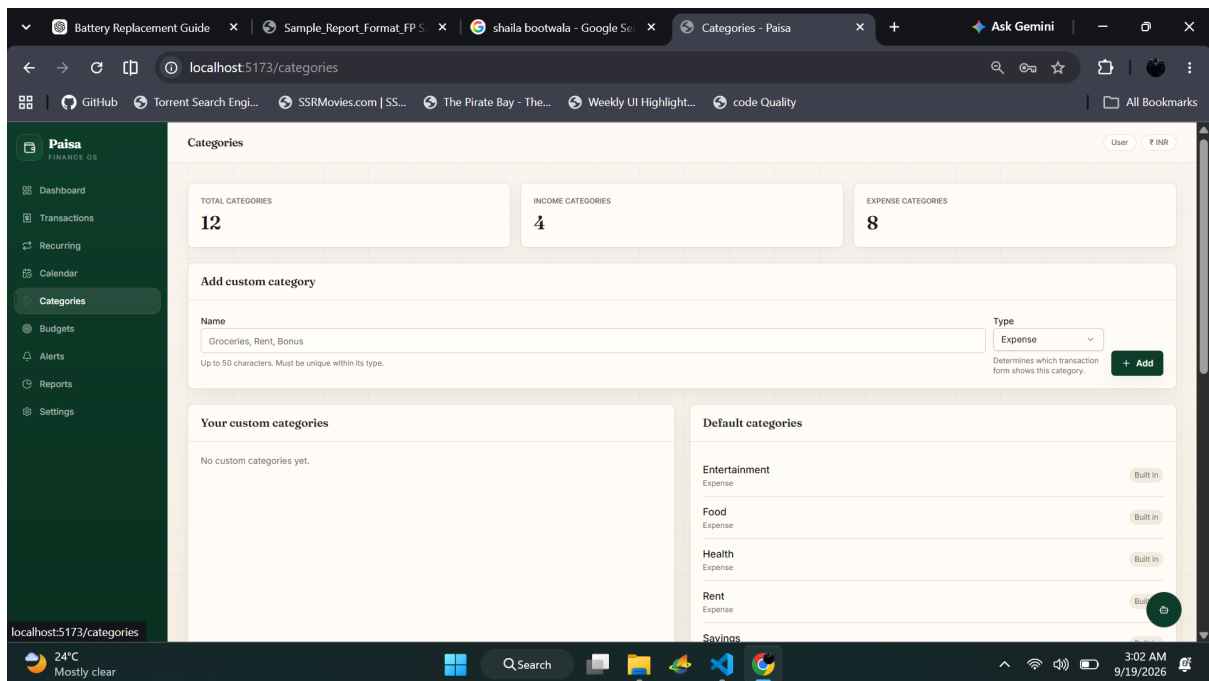


Figure 13.7: Budgets

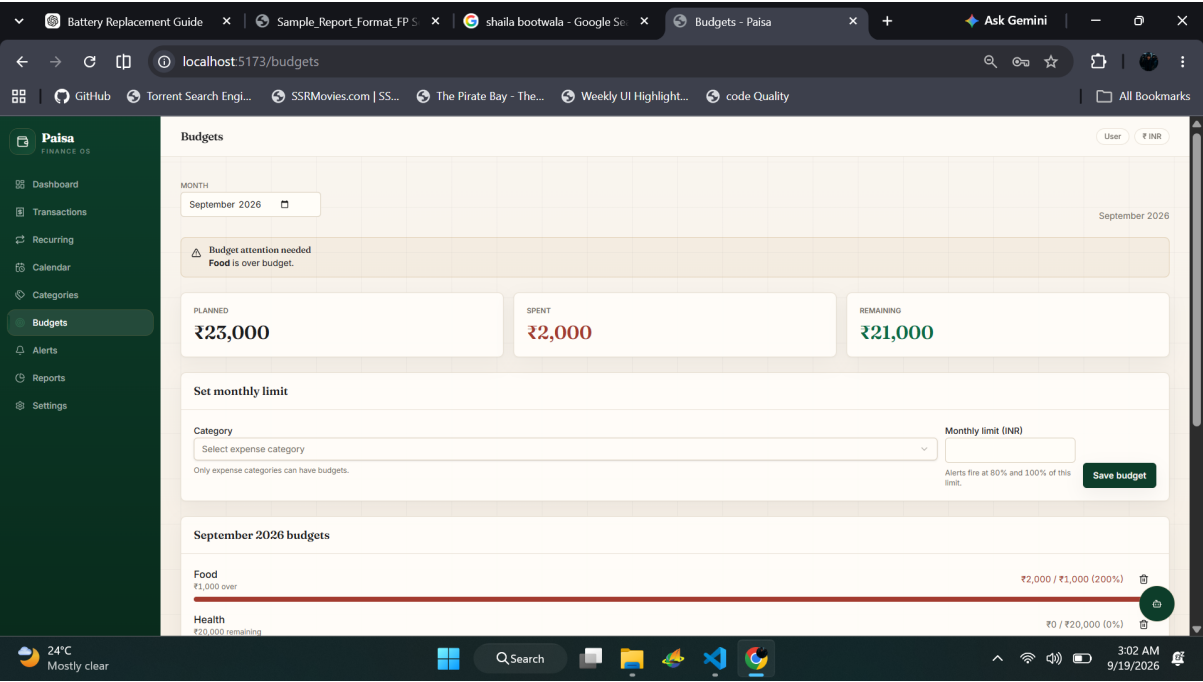
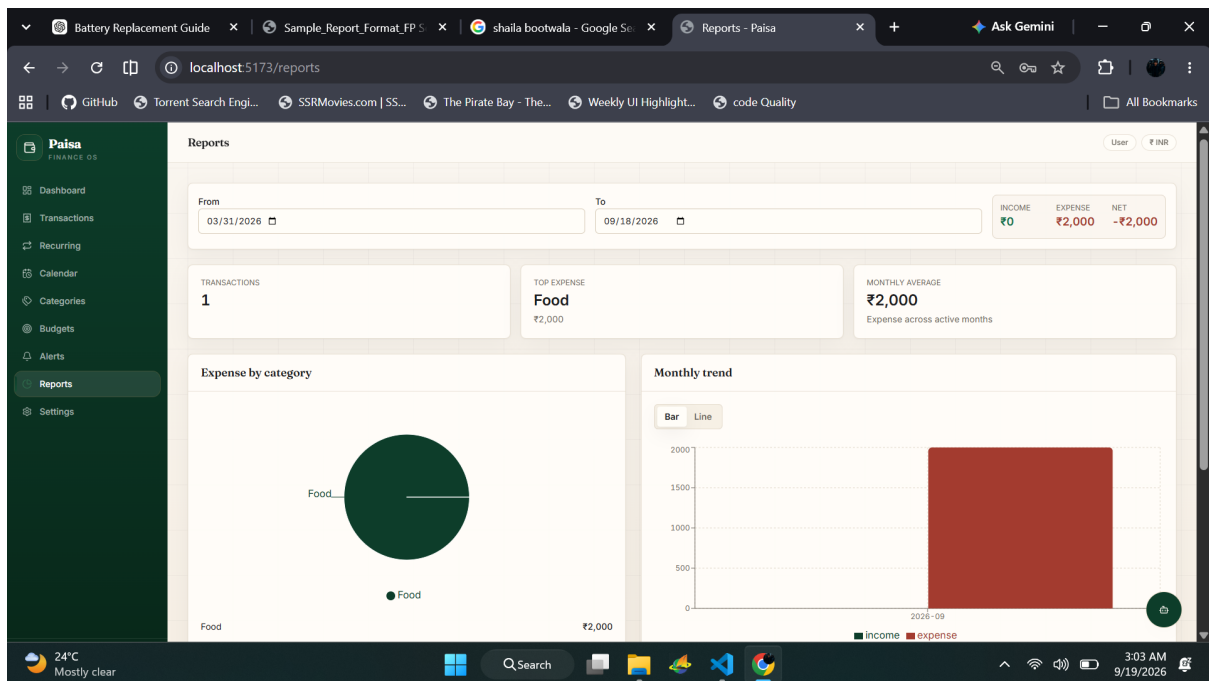


Figure 13.8: Reports



LIMITATIONS

Although PAISA fulfils its primary objectives of personal finance tracking and budget alerting, several limitations have been identified during development and testing:

- No direct bank/card sync: transactions must be entered manually or imported via CSV.
- The AI assistant is read-only — it can answer questions about existing data but cannot create or edit a transaction on the user's behalf.
- Budget alert detection runs on request rather than on a background schedule, so a user must open the app (or the client must call the detect-alerts endpoint) for a new breach to be recorded.
- The recurring-transaction materializer must be invoked (e.g. via a scheduled job or the client) rather than running as a persistent background worker.
- The system supports a single-user (non-shared) account model; there is no household or multi-user shared ledger.
- Automated test coverage and a CI/CD pipeline are not yet part of the project.

CONCLUSION

The PAISA project successfully demonstrates a functional, secure, full-stack personal finance tracker. By using a modern technology stack — React 19 and TypeScript on the frontend, Express 5 and Mongoose 9 on MongoDB for the backend — the application delivers categorised transaction tracking, monthly budgeting with automatic alerts, recurring schedules, multi-currency support, CSV import/export, and an AI assistant grounded in the user's own data.

While the system meets the requirements of a semester-level field project, it is a foundational implementation rather than a complete commercial finance platform. The identified limitations — manual entry instead of bank sync, on-request rather than background alert detection, and the lack of automated tests — outline clear directions for future work. Planned enhancements include a background job scheduler for alerts and recurring postings, shared/household accounts, and a native mobile client. Overall, PAISA establishes a solid, extensible foundation for digital personal finance management.

BIBLIOGRAPHY

1. React Documentation (v19) - <https://react.dev/>
2. TypeScript Documentation - <https://www.typescriptlang.org/docs/>
3. Vite Documentation - <https://vitejs.dev/>
4. TanStack Query Documentation - <https://tanstack.com/query/>
5. React Router Documentation - <https://reactrouter.com/>
6. Tailwind CSS Documentation - <https://tailwindcss.com/docs/>
7. shadcn/ui Documentation - <https://ui.shadcn.com/>
8. Express Documentation (v5) - <https://expressjs.com/>
9. Mongoose Documentation (v9) - <https://mongoosejs.com/docs/>
10. MongoDB Atlas Documentation - <https://www.mongodb.com/docs/atlas/>
11. JSON Web Tokens (JWT) - <https://jwt.io/introduction>
12. bcrypt.js Documentation - <https://www.npmjs.com/package/bcryptjs>
13. Google Gemini API Documentation - <https://ai.google.dev/docs>
14. Nodemailer Documentation - <https://nodemailer.com/>